

MaiAgent 技術文件

目錄

1. 生成式 AI 快速入門

- 1.1 大型語言模型 (LLM)
- 1.2 RAG 知識庫檢索系統
- 1.3 Embedding 模型
- 1.4 Reranker 模型
- 1.5 Parser 解析工具
- 1.6 圖像辨識支援

2. MaiAgent Work | 員工生產力整合

- 2.1 把 MaiGPT 加進您的系統
- 2.2 串接 MaiGPT：選對助理、補齊能力
- 2.3 成員記憶管理 API

3. MaiAgent Serve | 客戶服務整合

- 3.1 軟體廠商整合指南：把 AI Agent 嵌給您的客戶
- 3.2 Web Chat 嵌入與 SDK
- 3.3 MaiGPT 模式嵌入
- 3.4 注入頁面 Context
- 3.5 LINE LIFF：什麼是 LINE LIFF
- 3.6 LINE LIFF：如何串接
- 3.7 聯絡人 (Contact) 介紹及串接
- 3.8 聯絡人身份同步與 Token 更新
- 3.9 角色(Role) 和 聯絡人(Contact) 的差異
- 3.10 第三方客服系統串接（含轉真人）
- 3.11 對話可見權限與即時事件
- 3.12 對話交接與接續對話 API
- 3.13 Webhook

4. MaiAgent Build | AI 助理模組

- 4.1 角色指令
- 4.2 知識庫
- 4.3 FAQ 常見問題管理
- 4.4 回覆評估與監測結果
- 4.5 AWS Guardrails

5. MaiAgent Build | 進階生成式 AI 技術

- 5.1 Text to SQL
- 5.2 Function Calling
- 5.3 Skill 技能模組
- 5.4 Canvas
- 5.5 AI 安全防護機制
- 5.6 MCP (Model Context Protocol)
- 5.7 OAuth 2.0 整合與 Token 自動刷新機制
- 5.8 ICAP 內容驗證整合

6. MaiAgent Build | API 參考

- 6.1 快速上手
- 6.2 AI 助理列表
- 6.3 對話與訊息回覆 (串流/同步)
- 6.4 建立對話與訊息
- 6.5 Presigned 檔案上傳模式
- 6.6 對話附件管理 API
- 6.7 對話產出清單 API
- 6.8 使用知識庫—建立與互動
- 6.9 API 工具工程指南
- 6.10 自架 MCP Server 接入規格
- 6.11 站台品牌設定 API
- 6.12 Credits 項目顯示順序
- 6.13 內建工作流回覆模式退場

7. MaiAgent Build | AI Gateway

- 7.1 AI Gateway：OpenAI 相容 API 串接
- 7.2 AI Gateway 安全性

8. MaiAgent Build | 外部服務串接

8.1 Remote MCP 服務概述

8.2 Composio 串接

8.3 Zapier 串接

8.4 Google Sheet 串接

8.5 n8n 串接

9. 身份與權限

9.1 成員開通：SCIM／邀請／SSO

9.2 即時同步：在使用者事件中呼叫

9.3 名單同步工具：CSV／JSON／資料庫

9.4 SAML SSO 單一登入整合

9.5 OAuth 更新權杖的撤銷與輪替

9.6 知識管理權限(Query Metadata / 查詢元資料) 總覽

9.7 開始建構—使用「查詢構建器」

9.8 開始建構—使用 JSON 格式

10. 平台建置與部署

10.1 概覽

10.2 平台環境（SaaS/私有雲/地端）

10.3 部署架構

10.4 雲端模型推論 API 服務

10.5 GPU 算力硬體規劃

10.6 Celery 週期性任務配置與 OAuth Token 刷新

10.7 檔案庫 Office 線上編輯（ONLYOFFICE Docs 整合）

11. 其他

11.1 MaiAgent vs. Dify 比較

生成式 AI 快速入門

大型語言模型（LLM）

選擇的關鍵點

在選擇大型語言模型時，請考慮以下幾個關鍵因素：

環境（Environment）：使用環境是否能夠上網來決定使用雲端或地端模型。

品質（Quality）：模型生成回覆結果的能力與對於指令的服從性。

速度（Speed）：文字生成速度及對於延遲的要求，來確保模型的回應速度。

價格（Pricing）：各模型的 Credits 計價不同，請依使用需求與預算選擇，計價方式見 [Credits 計費制度](#)。

其他：是否支援多模態、是否支援 Function calling。

各模型的品質、速度與價格比較，可參考 [Artificial Analysis](#) 等持續更新的第三方評測。

MaiAgent 上支援的大型語言模型

MaiAgent 上架的模型會隨供應商發布與退役持續更新，本頁只列供應商與模型家族，不列具體型號。組織實際可選用的模型，以管理後台的「[模型調用權限](#)」頁為準：該頁列出 MaiAgent 為組織開通的模型，組織擁有者可在範圍內自行啟用或關閉。

閉源模型

模型供應商	模型家族	推論服務
OpenAI	GPT、o 系列	OpenAI、Azure、AWS Bedrock
Anthropic	Claude	AWS Bedrock、Google Vertex AI
Google	Gemini	Google Vertex AI
xAI	Grok	OpenAI 相容 API
Alibaba	Qwen（Max、Plus）	OpenAI 相容 API
Amazon	Nova	AWS Bedrock

開源模型

開放權重（open-weight）模型可以在 MaiAgent SaaS 上透過雲端推論服務使用，也可以在私有雲、地端以自建 GPU 部署，所需硬體請參考 [GPU](#) 章節。

模型供應商	模型家族	SaaS 上的推論服務
OpenAI	GPT-OSS	OpenAI 相容 API
Google	Gemma	OpenAI 相容 API
Alibaba	Qwen、Qwen-VL	OpenAI 相容 API
DeepSeek	DeepSeek	AWS Bedrock、Azure
Mistral AI	Mistral	OpenAI 相容 API
Moonshot AI	Kimi	OpenAI 相容 API
IBM	Granite	OpenAI 相容 API

地端自建時可部署的模型，以 MaiAgent 部署包的模型目錄為準，目前包含 GPT-OSS、Gemma、Qwen、Qwen-VL 等家族；部署規劃請洽 MaiAgent 窗口。

各供應商類型支援的 LLM、Embedding、Reranker 範圍不同，詳見[模型 API 推論服務](#)。

模型一定要 Fine-tune 嗎？

隨著人工智慧技術的快速發展，語言模型已經擁有了強大的語言理解和生成能力，並在許多領域中廣泛應用。例如，預訓練的語言模型可以輕鬆應對日常對話、文章生成以及簡單的問答任務。

然而，當模型面對更具挑戰性的專業領域任務，如醫療、法律或技術支持等領域時，僅僅依賴預訓練模型可能無法提供最佳的表現。有些開發者會選擇進行 **Fine-tuning**，也就是針對特定領域進行額外的訓練，以提高模型的專業知識。

然而，Fine-tuning 並不是唯一的解決方案，還有其他兩種有效的方法可以達成這一目標：**Prompt Engineering** 和 **RAG (Retrieval-Augmented Generation)**。

1. Prompt Engineering：通過精確的提示優化模型表現

Prompt Engineering 為設計精確的提示語句來引導模型生成所需的結果。這種方法的核心是根據任務需求，設計詳細表達方式，幫助模型縮小回答範圍，理解上下文和所需的輸出形式。

假設有一個語言模型，目的是根據用戶的需求推薦合適的產品。在這個情況下，如果表達「我想買一部高性能的手機」，可能會導致模型生成不夠精準的答案，因為「高性能」這個詞有很多不同的解釋，可能指的是處理速度、相機性能、電池壽命等。

為了提高推薦的精度，我們可以進行 **Prompt Engineering**，設計更加具體的問題或提供額外的上下文來引導模型更好地理解用戶的需求。例如，改為以下的提示語句：

"我需要一部具備**長時間電池續航**和**高效處理器**的手機，並且價格在 **500 美元到 800 美元**之間，請推薦幾款符合條件的手機。"

2. RAG：結合外部知識提升生成能力

RAG 是通過將外部知識檢索和生成過程相結合，來提升模型的表現。

在傳統的生成任務中，模型僅依賴預訓練時學到的知識。而 RAG 則利用檢索系統，即時獲取外部資料，並將這些資料與生成模型結合，從而更準確地回答問題或生成文本。

舉個例子，在醫療領域，當模型被問及某種罕見疾病時，RAG 可以先從專業醫學資料庫中檢索相關資料，再根據這些資料生成更準確的回答。這種方法的好處是，即便模型本身未在訓練過程中見過某些資料，它仍然能夠通過檢索現有知識來做出高質量的回應，特別適用於需要即時知識更新的情境，並能大大擴展模型的知識範圍。

有關 RAG 更詳細的介紹，將於下一章 [「RAG 知識庫檢索系統說明」](#)。

總結，Fine-tuning、Prompt Engineering 和 RAG 各有其優勢和適用範圍，可根據應用場景和需求選擇最合適的策略，而不必僅依賴於單一的 Fine-tuning 方法。

Prompt Engineering 提供了一種成本低且靈活的解決方案，通過設計精確的提示語句來引導模型產生高質量的結果。

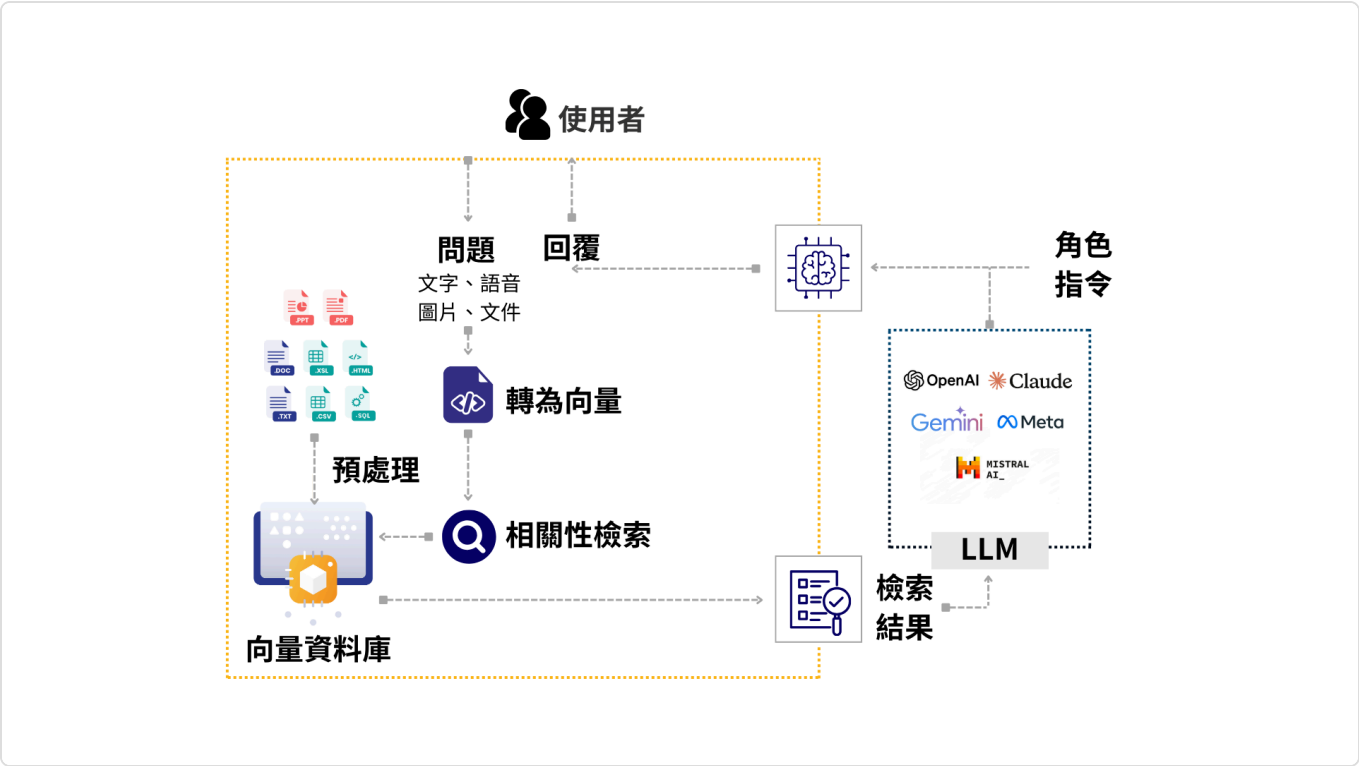
而 **RAG** 則提供了一種結合外部知識和生成能力的方法，能夠在需要動態獲取知識的情況下，提供更精確的答案。

Fine-tuning 可以顯著提升模型在特定領域的表現，但需要大量專業資料並消耗額外資源，可視為 **Prompt Engineering** 與 **RAG** 兩種方式都無法成功，才實行的最後手段

RAG 知識庫檢索系統

利用外部資料庫或知識庫進行檢索，並將檢索結果與大型語言模型結合，生成內容更加精準且上下文相關的回應。

RAG 技術的核心是將生成式 AI 的語言能力與知識檢索能力結合，使模型在回答問題時，不僅依賴於內部訓練數據，還能動態地從外部資料庫中獲取最新且更具專業性的資訊，並將這些資訊融入到生成的回應中。



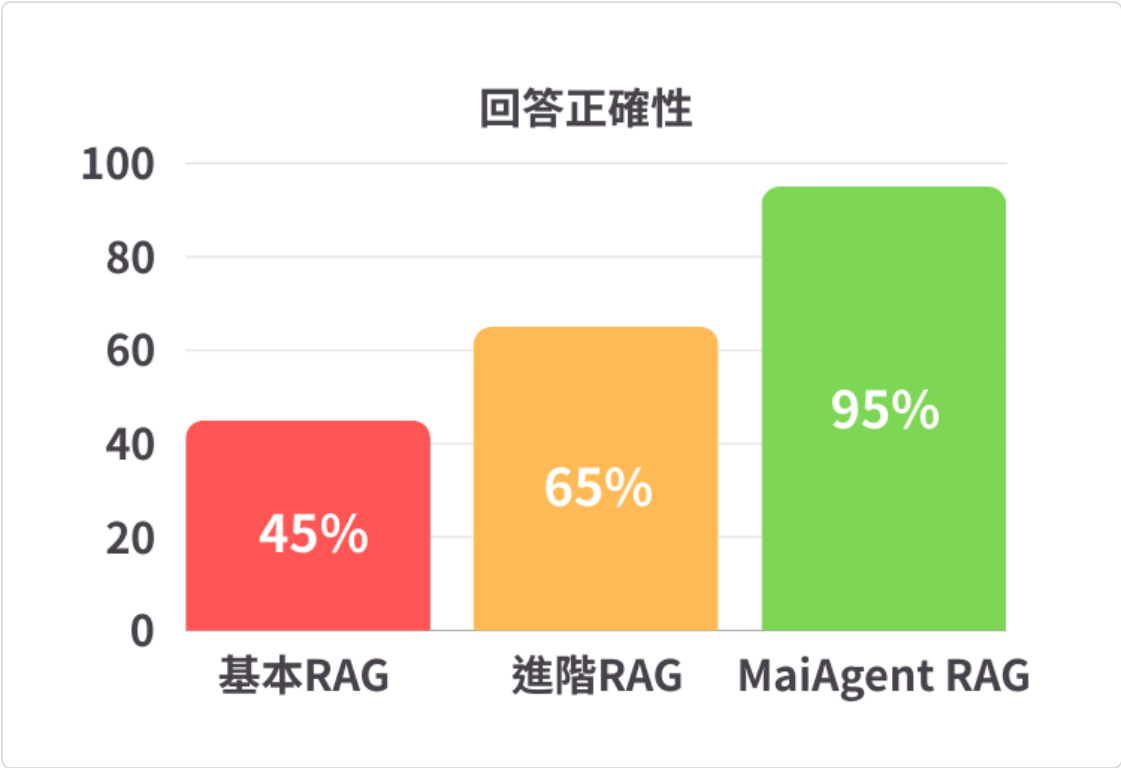
RAG 流程

高精準度的 RAG 系統

RAG 知識庫檢索系統雖然能夠快速使用向量搜尋（Vector Search）來實現，並推出基礎版本，但要進一步提升其回覆精準度卻具有挑戰性。回覆精準度對於使用者體驗至關重要，因為它直接影響到使用者對系統回應的信任度與滿意度。如果回覆精準度不足，使用者可能會對系統的答案產生懷疑，從而降低使用意願。

根據 2023 OpenAI 開發者大會資料顯示，RAG 系統若僅作單純的向量相似度搜尋（Vector Search）並且選擇正確的內嵌模型（Embedding Model），能夠達到 45%。加上 HyDE Retrieval, FT Embeddings, Chunk/Embedding Experiments 可以達到 65% 的回覆精準度。

MaiAgent RAG 除了包含 OpenAI 開發者大會中所提到的 RAG 技術以外，亦結合各種經典 NLP 演算法與獨家的檢索技術。透過內部資料集與 OpenAI RAG 的回覆正確性相比，兩者皆能達到 95% 的回覆精準度。



MaiAgent RAG 回覆精準度

MaiAgent 平台提供兩種 RAG，分別是 MaiAgent RAG 與 OpenAI RAG，以下為各面向的比較表：

	MaiAgent RAG	OpenAI RAG
模型支援度	支援所有模型👍	僅支援 OpenAI 模型
環境支援度	支援雲端、地端👍	僅支援雲端，資料需要送至 OpenAI
回覆精準度	超高👍	超高👍
支援檔案格式	支援所有常見格式👍 doc, docx, xls, xlsx, csv, ppt, pptx, pdf, txt, json, jsonl, md	不支援 xlsx, csv 不支援 jsonl 不支援舊版 Office 檔案（doc, xlsx, ppt）
支援文件中的圖片	有（目前為實驗性功能）👍	無
支援文件中的表格	有（目前為實驗性功能）👍	無
支援對話中的附件上傳	支援👍	支援👍

	MaiAgent RAG	OpenAI RAG
資料切片透明度	可視化👍	黑盒子
Debug 難度	一般👍	黑盒子，無法 Debug
Top K 調整	企業版客製化功能👍	無
切換 Embedding 模型	企業版客製化功能👍	無

Embedding 模型

什麼是 Embedding？

在 RAG (Retrieval-Augmented Generation) 處理過程中，**Embedding** 的核心任務是將經由 Parser 處理後的大量文本資料，例如文檔、知識庫內容，轉換成電腦能夠理解和比較的數值形式——即向量（vectors）。

這個轉換過程使得 RAG 系統能夠：

1. 理解語義

Embedding 模型捕捉文字的深層語義含義，而不僅僅是表面詞彙。這意味著即使查詢和文檔中的措辭不同，只要語義相近，系統也能識別它們的關聯性。

2. 高效檢索

將文本轉換為向量後，系統可以利用高效的向量相似度搜索算法，快速從龐大的資料庫中找到與用戶查詢最相關的文本片段。

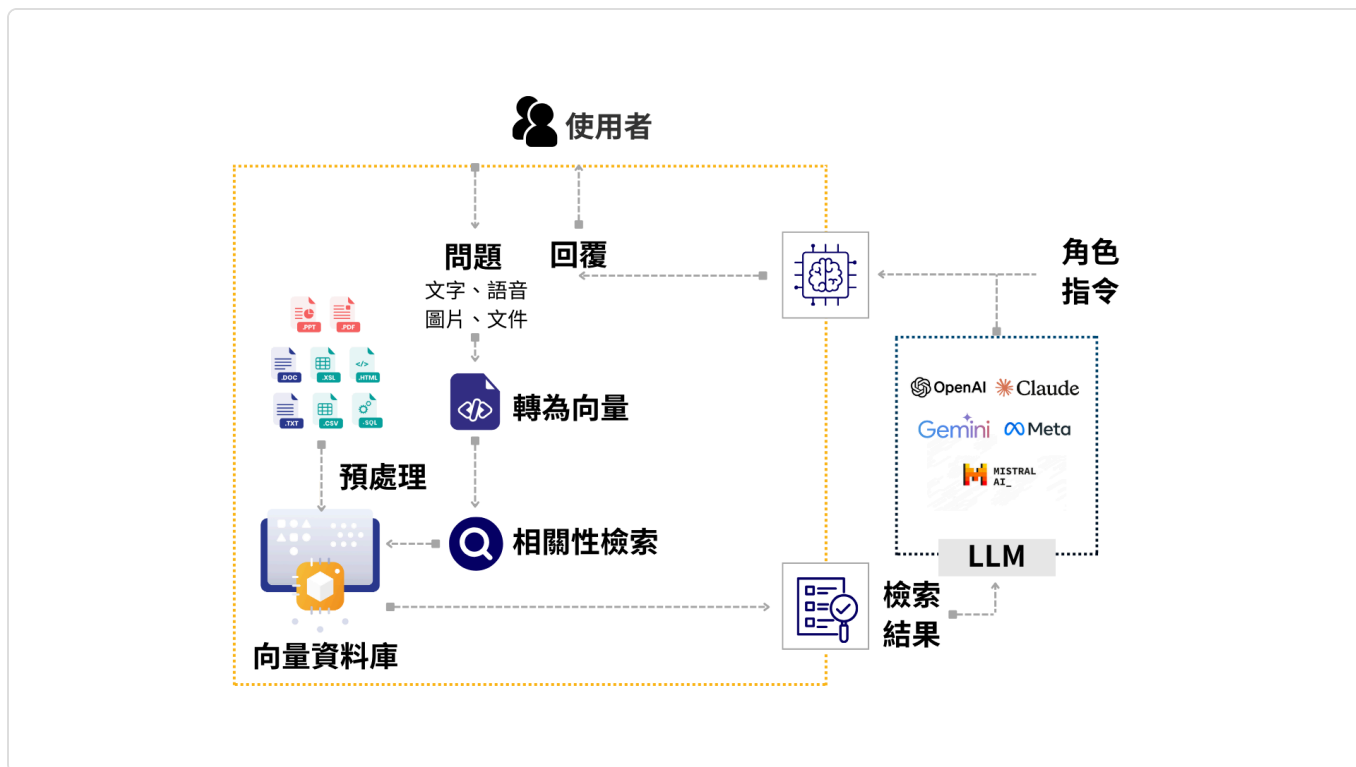
3. 提升生成質量

檢索到的相關文本片段會被提供給大型語言模型（LLM）作為上下文參考，從而幫助 LLM 生成更準確的回答。

Embedding 在 MaiAgent 的 RAG 技術中發揮兩項關鍵作用

1. 將知識庫資料內容轉換為向量，並儲存在向量資料庫中

2. 將使用者的問題向量化，與資料庫中向量進行相似度比對，找出最相關內容



RAG 流程

簡單來說，Embedding 是 RAG 系統的基石，它將非結構化的文本資料轉化為可計算、可比較的向量，是實現精準資訊檢索和高質量內容生成的關鍵前置步驟。

Embedding 對 RAG 系統的影響

1. 檢索質量

語義理解深度：高質量的 Embedding 模型能夠更準確地捕捉文本的語義內涵，提升檢索的相關性

上下文感知：優秀的 Embedding 能夠理解文本的上下文關係，確保檢索結果的連貫性

多語言支援：強大的多語言 Embedding 模型能夠處理跨語言的知識檢索需求

2. 系統性能

檢索速度：Embedding 模型的向量維度和計算效率直接影響檢索響應時間

資源消耗：不同 Embedding 模型對計算資源的需求不同，影響系統的擴展性

並行處理：高效的 Embedding 模型能夠支援大規模並行檢索

MaiAgent 支援的 Embedding 模型

模型名稱	模型開發商	產地	特性	開源	部署方式	MTEB 平均成績
Cohere Embed v4.0	Cohere	加拿大	支援多語言、性能最高	否	需使用雲端 API 推論服務	尚未公開，可參考 v3.0 為 64.47
Cohere Embed Multilingual v3.0 (Bedrock)	Cohere	加拿大	支援多語言、性能高	否	需使用雲端 API 推論服務	64.47
OpenAI text-embedding-3-Large	OpenAI	美國	多語言（尤其在英語語境下表現強大）、性能中等	否	需使用雲端 API 推論服務	64.68
EmbeddingGemma	Google	美國	開源、支援多語言、輕量	是	可針對需求部署於雲端或地端 GPU	61.15
Mxbai-embed-large	Mixedbread AI	美國	開源、效能與資源平衡佳；長上下文表現強	是	可針對需求部署於雲端或地端 GPU	64.68
BGE-Large	BAAI	中國	開源、支援多語言、輕量	是	可針對需求部署於雲端或地端 GPU	64.23
Nomic-embed-text	Nomic AI	美國	開源、輕量	是	可針對需求部署於雲端或地端 GPU	62.39
Qwen3-Embedding 0.6B	Alibaba	中國	開源、輕量	是	可針對需求部署於雲端或地端 GPU	61.82
Granite-embedding-278m-multilingual	IBM	美國	開源、多語言、輕量	是	可針對需求部署於雲端或地端 GPU	56.1

為確保語意表示的精確性與檢索準確度，MaiAgent 所提供的 Embedding 模型將依據 MTEB（Massive Text Embedding Benchmark）標準進行選型與評估。MTEB 為目前主流的語意向量化模型比較基準，涵蓋多種任務類型，包含：Retrieval（語意檢索）Classification（分類）Clustering（聚類）Reranking（重排序）STS（Semantic Textual Similarity 語意相似度）Summarization QA Pair Classification 等

MaiAgent 的 Embedding 技術優勢

1. 模型選擇靈活：

提供多種 Embedding 模型選擇，滿足不同需求

2. 部署方式多樣：

支援雲端和本地部署，確保數據安全

3. 性能優化：

針對 RAG 場景進行特別優化，提供最佳檢索效果

4. 成本效益：

根據實際需求選擇合適的模型，平衡性能和成本

Reranker 模型

什麼是 Reranker ？

Reranker 是一種用於**排序任務**的模型或系統，通常在信息檢索、推薦系統或自然語言處理（NLP）中使用。其主要目的是在一組候選結果中重新排序，以便返回最相關或最合適的選項。

Reranker 的運作方式

通常，當一個搜索引擎或推薦系統返回一系列候選項（例如搜索結果、推薦商品或文章），已經根據某些初步的排序標準進行了排序。這些初步排序可能基於簡單的匹配或基本的相關性指標。然而，這樣的排序並未完全考慮所有的細節或更深層次的語義關係，可能導致不完全符合用戶需求的結果。

例如，假設你在網路上搜尋“最好吃的披薩店”，搜尋引擎會根據一些基本標準給你返回很多結果，比如餐廳的名稱、評分、地址等。這些結果可能已經根據關鍵字或簡單的匹配進行排序了，但它們不一定完全符合你的需求。

這時 Reranker 會進一步幫助你“**優化排序**”，會根據更多的細節來檢查這些結果，可能會考慮你過去的搜尋習慣、其他人對該餐廳的評價，或者餐廳的最新狀況等，從而重新排列結果，最後挑出最相關的餐廳，讓你能夠更快找到最佳選擇。以這個範例而言，Reranker 的運作流程如下：

初步排序：使用簡單的排序模型（如基於關鍵字匹配、TF-IDF 或其他特徵）對候選項進行初步排序

搜尋：「最好吃的披薩店」，搜尋引擎返回了一堆餐廳的列表

候選結果篩選：根據更多特徵（如上下文理解、語義關聯、用戶行為等）來細化排序，重新對候選項進行評分

再仔細看看這些餐廳，考慮是否有餐廳在你常去的區域，或是否有其他用戶的高評價，或餐廳提供的優惠等等

最終排序：基於其評分結果返回最相關或最合適的結果

Reranker 會將最符合你需求的餐廳排在最前面，讓你更容易找到最適合的選擇

MaiAgent 提供的 Reranker 模型

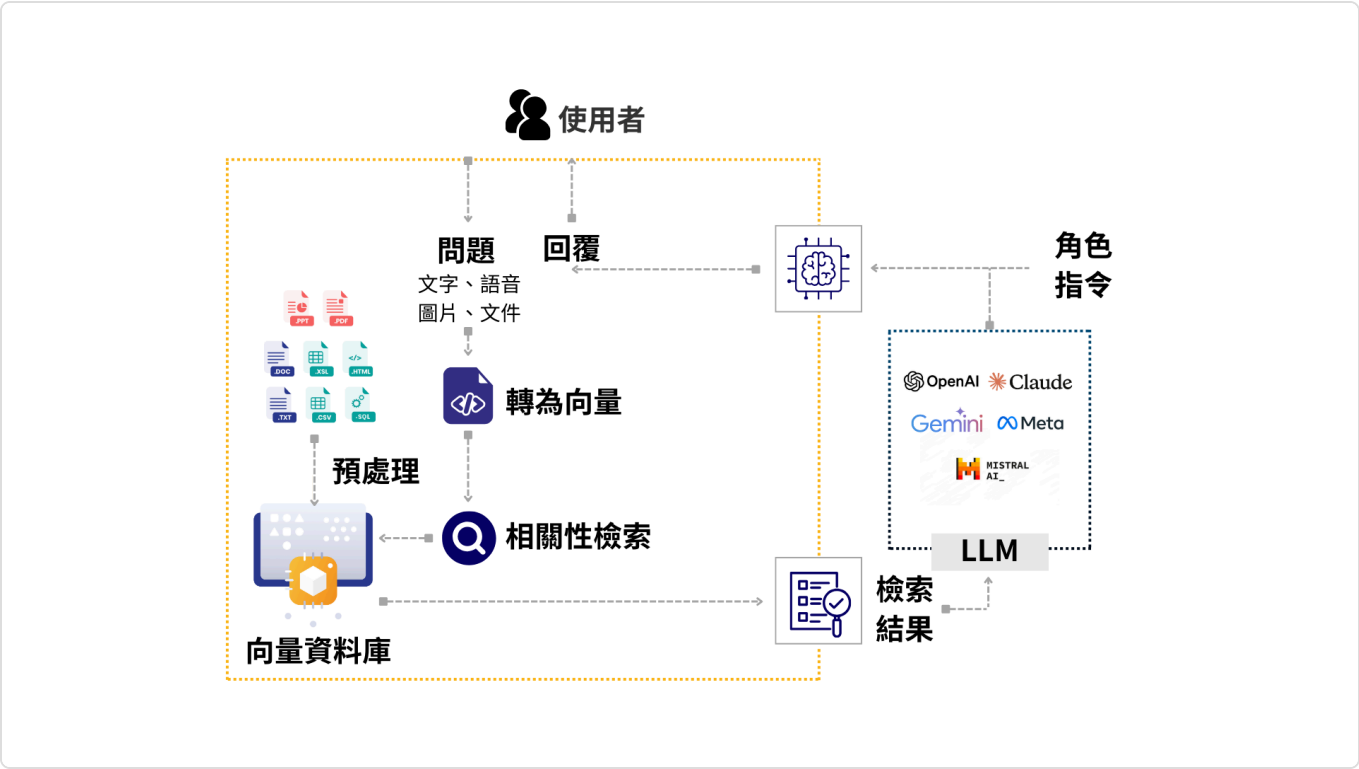
Cohere Rerank v3.5 的特色

Cohere Rerank v3.5 是 Cohere 提供的一款 **重新排序（reranking）** 模型，專門用來優化搜尋結果或候選答案的排序，能在候選結果中進行精細排序，根據更深層次的語義理解與上下文來選擇最相關的結果，從而提升最終的品質。

Parser 解析工具

什麼是 RAG Parser ？

RAG Parser 是 Retrieval-Augmented Generation (RAG) 系統中的關鍵步驟，負責解析與拆解原始資料，作為 Embedding 向量化處理的前置步驟，為後續的向量化與語意檢索提供基礎，對整體資料品質與檢索效果有決定性影響。



RAG 流程

一、文件解析類 Parser

MaiAgent 提供四種文件解析器，適用於 PDF、Word、Excel、圖片等多種文檔格式：

功能特性	MaiAgent Parser (預設)	MaiAgent Parser (Online)	MaiAgent Parser (Offline)	Vision Parser
價格成本	低	高	中	高
圖片內容解析效果	無法解析圖片中的文字	僅 OCR 可解析圖片中的文字	OCR + AI 理解圖片語意	AI 視覺理解，效果最佳
使用 LLM	否	是	是	是

功能特性	MaiAgent Parser (預設)	MaiAgent Parser (Online)	MaiAgent Parser (Offline)	Vision Parser
文字解析效果	標準	好	好 (結構保留佳)	好
表格解析	原生提取	AI 智慧提取圖片裡面的文字	原生結構保留 (最佳)，除了視覺理解外還包含圖片的靜態資源	AI 視覺辨識將圖片產生視覺理解後的文字內容
解析時間	快	中	慢	慢
落地 (無網路)	可以	不行	可以，但需部署 VLM	不行
支援格式數	22 種	20 種	20 種 (僅 PDF 支援 OCR)	7 種

「自動偵測 (預設)」與解析計費

上傳 PDF、Word (.docx)、PowerPoint (.pptx) 到知識庫時，上傳視窗每個檔案旁的解析器下拉選單預設是 **自動偵測 (預設)**，不是固定使用 MaiAgent Parser。系統會在上傳後讀取檔案結構再決定解析器：

只有文字、沒有明顯圖片內容的檔案 → 使用該檔案類型的預設解析器 (一般為 MaiAgent Parser)
掃描檔 (沒有文字層)、含圖片內容的檔案 (例如工程圖面、掃描件) 與 PowerPoint → 系統從支援 OCR 或視覺理解的解析器 (MaiAgent Parser (Online)、MaiAgent Parser (Offline)、Vision Parser) 中挑選；找不到合適的解析器時使用預設解析器

判斷依據是檔案結構 (有沒有文字層、含圖片與含表格的頁數)，不是圖片張數多寡；Word 檔只要內嵌圖片就視為含圖片內容。因此只有少量圖片的文件仍可能改用上述解析器，實際結果以知識庫文件列表 **解析器** 欄顯示的解析器為準。點數依**實際完成解析的那個解析器**計算：

解析器	計費方式	交易記錄 顯示
MaiAgent Parser	依解析後的段落數 (chunks) 計費	Document parsing (maiaagent): 檔名
MaiAgent Parser (Online)	依實際計費頁數計費，每次解析都計費	Document parsing (llama): 檔名
MaiAgent Parser (Offline)	依解析後的段落數 (chunks) 計費	Document parsing (docling): 檔名
Vision Parser	依該解析器設定模型的輸入／輸出 token 計費	Vision parsing input: 檔名、Vision parsing output: 檔名

檔案處理完成才扣款，處理失敗不扣款。同一個檔案，每一種解析器的費用各最多只收一次（所有 Vision Parser 合併算同一種）。在文件列表換成該檔案沒用過的解析器，會重新解析並依新解析器計費；換回先前用過的解析器、用同一個解析器重新解析、或在不同的 Vision Parser 之間切換，都不會再扣點。例外：MaiAgent Parser (Online) 依實際計費頁數收費，每次解析（含換回它或用它重新解析）都會再計費一次；解析服務命中快取、實際未計費時不扣點。先前解析已扣的點數一律不退回，因此同一個檔案的交易記錄可能同時出現多種解析項目。在文件列表更換解析器（含 [批量修改解析器](#)）會立即重新解析，沒有確認視窗。要固定解析成本、避免自動改用其他解析器，請在上傳時直接選擇 MaiAgent Parser（或其他指定的解析器），不要使用 [自動偵測（預設）](#)。

二、語音轉文字類 Parser

MaiAgent 提供四種語音轉文字解析器，可將音訊檔案轉錄為文字後納入知識庫：

功能特性	Azure Speech	Whisper (Groq)	Whisper (OpenAI)	Whisper (Offline)
價格成本	高	低	中	免費
轉錄準確度	高	高 (large-v3)	高 (whisper-1)	高 (視本地模型)
使用 LLM	否	否	否	否
解析速度	即時	最快	中	視硬體而定
落地（無網路）	不行	不行	不行	可以
多語言支援	是	是（自動偵測）	是（自動偵測）	是（自動偵測）
自訂提示詞	否	是	是	是
VAD 語音偵測	—	是	是	是
資料隱私	雲端 (Azure)	雲端 (Groq)	雲端 (OpenAI)	完全本地

音訊檔案上傳至知識庫後，系統會自動進行語音轉文字解析，並提供逐字稿檢視與下載功能：

MAIAGENT AI 功能 / 知識庫 / 設定 Ask AI 文件 林群峰 思邁智能...

思邁智能股份有限... 基本 標籤管理 < 返回 專案執行計劃書審查會議.mp3 檢視文件

專案執行計劃書審查會議.mp3

音訊檔案

0:00 / 2:36:41

逐字稿 純文字 下載

00:00 - 00:07
還是請廠商先就那個整個的專案計畫先說明一下

00:07 - 00:07
好

00:07 - 00:09
然後我們再審

00:09 - 00:10
OK啊

00:10 - 00:10
對啊

00:10 - 00:11
看你啊

00:11 - 00:11
OK啊

00:11 - 00:13
也不用太拘束啦

字數 2340 命中次數 0 字數 2233 命中次數 0 字數 2169 命中次數 0

即時轉錄 客服對話

逐字稿檢視：顯示時間戳與對應文字內容



下載逐字稿：支援 TXT（純文字）與 SRT（字幕格式）兩種格式

圖像辨識支援

什麼是 VLM (Vision Language Model) ?

視覺語言模型 (Vision Language Model, VLM) 是一種先進的人工智慧模型，它能夠理解圖像內容並將圖像資訊與文字資訊相結合。不同於傳統的圖像處理技術，VLM 不僅僅是「看見」圖像，更能「理解」圖像中的物件、場景、關係，並能根據圖像內容進行文字生成、回答問題或執行相關指令。

VLM 的核心能力在於其多模態處理能力，即同時處理和理解來自不同來源（視覺和文字）的資訊。這使得 VLM 能夠執行更複雜和更具上下文感知能力的任務。

VLM 與傳統 OCR 的比較

傳統的光學字元辨識 (OCR) 技術專注於從圖像中偵測並提取文字，但在理解圖像的整體語義和非文字內容方面存在局限性。

特性	傳統 OCR	VLM (Vision Language Model)
主要功能	從圖像中提取文字	理解圖像內容，並將其與文字資訊結合
資訊處理	單模態 (僅處理文字像素)	多模態 (同時處理圖像視覺資訊和文字語義)
理解層次	字元級辨識	場景理解、物件識別、關係推斷、上下文感知
主要任務	文件掃描、文字擷取	圖像描述生成、視覺問答 (VQA)、圖像檢索、物件偵測等
上下文感知	有限，主要依賴後處理中的語言模型	強，能理解圖像的整體情境和細節
處理非文字內容	通常忽略或無法處理	能夠識別和理解圖像中的物件、場景、動作等

VLM 的優勢在於：

- * **更深層次的理解**：VLM 不僅讀取文字，還能理解圖像的語義內容，例如辨識物件、分析場景、理解圖像中元素之間的關係。
- * **互動性**：VLM 可以進行視覺問答 (Visual Question Answering, VQA)，即根據圖像內容回答用戶提出的問題。
- * **內容生成**：VLM 可以為圖像生成描述性文字 (Image Captioning)。
- * **多功能性**：除了文字相關任務，VLM 還能應用於更廣泛的視覺理解任務。

VLM 實際應用案例

VLM 的強大功能使其在多個領域都有廣泛的應用前景：

視覺問答 (VQA)

應用：用戶上傳一張圖片，並提問關於圖片內容的問題，例如「圖片中的人穿著什麼顏色的衣服？」或「這張照片是在室內還是室外拍攝的？」。

場景：智慧助理、教育、視障輔助。

圖像描述生成 (Image Captioning)

應用：自動為圖片生成簡潔、準確的文字描述。

場景：自動化圖片標註、內容管理系統、社交媒體內容生成、輔助視障人士理解圖片。

內容導向的圖像檢索

應用：允許用戶使用自然語言描述來搜索圖片，例如「查找辦公室裡有人在開會的圖片」。

場景：大型圖片庫管理、電子商務產品搜索。

多模態資料分析

應用：結合醫學影像和病歷文字，輔助醫生進行診斷；分析產品圖片和用戶評論，進行市場趨勢預測。

場景：醫療、零售、金融。

人機互動

應用：讓機器人或虛擬助理能夠理解其視覺環境，並據此與人進行更自然的互動。

場景：智慧機器人、自動駕駛汽車（理解交通標誌和路況）。

MaiAgent 圖像辨識的優勢

MaiAgent RAG 整合 VLM 技術，為技術人員和開發者提供了便捷高效的圖像辨識與理解解決方案，具備以下優勢：

附件圖片 VLM 辨識（圖像理解與問答）

MaiAgent RAG 能夠對用戶上傳的附件圖片進行 VLM 辨識，提供精準的圖像內容理解

附件文件內嵌圖片的 VLM 辨識

MaiAgent RAG 正在積極開發對附件文件（如 PDF、Word 文檔）內部嵌入圖片的 VLM 辨識功能。這意味著系統不僅能理解文檔中的文字，更能解析圖片內容，實現真正的多模態文檔理解。這是許多標準 RAG 系統所不具備的進階功能。

附件文件圖片的內容問答

無論是附件圖片，還是文檔中嵌入的圖片，MaiAgent RAG 均能支援基於圖片內容的問答，用戶可以直接針對圖像細節提問，獲得精準的回答。

知識庫文件圖片的內容問答

MaiAgent RAG 支援知識庫文件中的圖片問答（需要在角色指令下 Prompt，以 Markdown 格式顯示圖片）

知識庫文件內的圖片 VLM 辨識後問答

MaiAgent 推出對知識庫文件內的圖片先進行 VLM 辨識，再結合辨識結果進行深度問答的實驗版功能。這將進一步打通圖像資訊與知識庫的連接，提供更全面的知識服務。

透過 MaiAgent 的 VLM 技術，您可以更深入地挖掘圖像資訊的價值，實現更智慧化的人機互動和自動化流程。

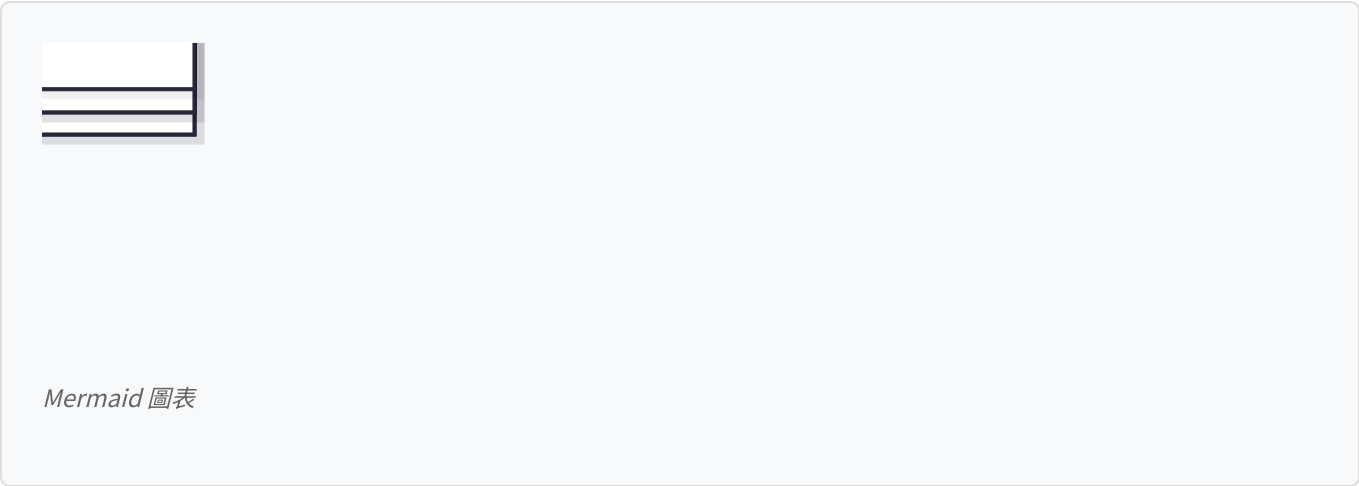
MaiAgent Work | 員工生產力整合

把 MaiGPT 加進您的系統

這一頁給「我們的系統要加上 MaiGPT」的工程師從頭讀到尾。不管您是企業自己的入口網站、內部工作台，還是替客戶代管的 SaaS，只要使用者是 **MaiAgent 組織的成員**（員工、承辦人），流程都一樣。每一步把要做的事直接寫在這裡，想深入才點連結。

如果您的使用者是您的客戶、會員或匿名訪客、不是組織成員，MaiGPT 產品本身不能給他們用，請改走 自建 AI Agent+ MaiGPT 視窗模式，聯絡人身份對接見軟體廠商整合指南。

全貌



做完之後：員工在您的系統登入，頁面裡直接出現 MaiGPT，能生圖、Deep Research、用連接器，權限、額度、對話歷史都跟著他在 MaiAgent 的成員身份；他不需要另外登入 MaiAgent，也不需要知道 MaiAgent 的存在。

開始前準備

項目	到哪拿
一個 MaiAgent 組織，方案含 MaiGPT 與身份同步	不確定請洽業務
組織 API Key，持有者是組織擁有者或具「組織總覽」管理權限	取得方式見 快速上手
MaiGPT 的 Web Chat ID	<code>GET /api/v1/web-chats/</code> 回傳中名為 <code>MaiGPT</code> 、 <code>accessType</code> 為 <code>web</code> 的那一筆

項目	到哪拿
角色先建好、權限先設好	後台「組織設定 → 組織總覽 → 編輯角色權限」建例如「員工」角色，勾選 MaiGPT 使用權（ <u>設定方式</u> ）。 成員的能力全部來自角色 ，這步沒做，後面全部白做
決定成員從哪來	下一節三選一

第一步：開通成員

您的使用者要先是 MaiAgent 組織的成員。三種做法，選最省事的那個：

組織裡已經有人在用 MaiAgent？接 SCIM 不會讓他們原本的帳號失效，也不會建立重複帳號；被接管的既有成員之後會跟著您的系統同步。細節見既有帳號與成員會怎樣。

您的系統有使用者管理（推薦）

在您系統新增、修改、停用使用者的事件裡，順手呼叫 MaiAgent 的 SCIM 端點，成員即時一致。

後台「組織設定 → 身份同步」選 SCIM，「連線資訊」分頁複製 Endpoint URL 與 Token（Token 只顯示一次，放伺服器環境變數）

把即時同步頁的 `MaiAgentMembers` client 放進您的專案，在事件裡呼叫：

```
members = MaiAgentMembers()    # 讀環境變數 MAIAGENT_SCIM_URL MAIAGENT_SCIM_TOKEN

# 新增使用者
user.maiagent_member_id = members.on_user_created(
    external_id=user.id, email=user.email, name=user.display_name, groups=["員工"],
)
# 改名、換部門
members.on_user_updated(user.maiagent_member_id, user.id, user.email,
user.display_name, ["員工"])
# 停權、離職
members.on_user_deactivated(user.maiagent_member_id)
```

第一次上線先把既有使用者跑一輪 `on_user_created`（冪等，批次補建），之後靠事件維持。

只有名單（HR 匯出、Excel、資料庫）

用名單同步工具：準備一份四欄的 CSV 或 JSON（人員編號、Email、姓名、角色），設好 SCIM 連線環境變數，先 `--dry-run` 看計畫再真跑，之後排程每天跑一次。名單上有的建立或更新、名單上消失的停用。

有 Okta／Entra ID／JumpCloud

把 SCIM 的 Endpoint URL 與 Token 填進 IdP 的 provisioning 設定，IdP 會自動推送使用者與群組。細節見從 IdP 推送。

三件事要記得，都是實際整合時最常卡的：

角色名稱對上後台建好的角色：SCIM 建成員時 `groups` 填的名稱，就是後台那個已設好權限的角色；名稱對不上會自動建一個**沒有權限**的新角色。

Email 是帳號的鍵：`userName` 就是之後換發 token 用的 Email；建立後不能經 SCIM 改 Email。

SCIM 建的帳號沒有密碼：這條路的員工不會登入 MaiAgent，這正是我們要的；若同時要開放 MaiGPT 網頁版或 App，另設 SSO。

第二步：後端換發成員 token

在您的後端開一支只給**已登入使用者**呼叫的端點（例如 `GET /maigpt-token`），內部以組織 API Key 向 MaiAgent 換發該成員的短效 token：

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
-H "Authorization: Api-Key <您的組織 API Key>"
-H "Content-Type: application/json"
-d '{"email": "alice@acme.com}"'
```

成功回 `201`：

```
{
  "accessToken": "<成員短效 JWT>",
  "organizationId": "8d2f ... - ...",
  "expiresAt": 1789906401000,
  "member": { "id": "c1a4 ...", "userId": "5e77 ...", "email": "alice@acme.com",
  "name": "Alice" }
}
```

把整份回應原樣回給前端就好。規則與狀態碼：

事實	說明
效期 15 分鐘、不發 refresh token	續期由前端 SDK 自動處理（第三步），您的後端只要「被叫到就換一顆」—— 每次都換新的，不要快取
token 只能用在 MaiGPT 對話端點	拿去打 <code>/api/v1/</code> 任何 API 一律 <code>401</code> ，外洩也不能碰組織資料

事實	說明
401 / 403	沒用 Api-Key / Key 的持有者不是擁有者或組織管理員
404	這個 Email 不是本組織的啟用成員—— 第一步沒做、或已被停用
每次換發都留稽核紀錄	組織的 API 請求紀錄可查，回應中的 token 不會被記錄

參數與更多細節見[成員 token 換發](#)。

第三步：前端嵌入

在要放 MaiGPT 的頁面加入嵌入碼，`auth.getToken` 指向第二步的端點；其餘都交給 SDK（初始化時取一次 token、到期前自動再取、同一則對話續用）：

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPT 的 Web Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',           // 選填：左上角品牌名
    locale: 'zh-TW',                  // 選填：介面語系
    auth: {
      getToken: async () => {
        const res = await fetch('/maigpt-token', { credentials: 'include' })
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

`sourceId` 一定要填回應裡的 `member.userId`，不是 `member.id`、也不是您系統的使用者編號。MaiGPT 靠它把對話對應到成員，填錯會變成匿名對話、看不到該成員的歷史。

容器要自己給高度；SPA 要等容器 mount 後再載入 SDK；`baseUrl` 與 SDK 網址是 SaaS 位址，私有部署請換成您的網域。版面與其他參數見[MaiGPT 模式嵌入](#)。

第四步：登出與離職

事件	您要做的事	結果
使用者登出您的系統	前端呼叫 <code>MaiAgent.auth.signOut()</code>	iframe 清掉身份並停止續期；已發出的 token 最多再存活 15 分鐘
使用者離職／停權	第一步的停用（ <code>on_user_deactivated</code> 、名單移除、IdP 停用）	之後換發回 <code>404</code> ，嵌入頁自然失效；對話與額度紀錄保留
復職	第一步的重新啟用	角色與歷史恢復

第五步：驗收

依序確認，每一項都有明確的判斷點：

成員存在且有角色：後台「成員管理」看得到這個人，角色是您設好權限的那個。

換發成功：第二步 curl 回 `201`；換一個不存在的 Email 回 `404`。

能對話：嵌入頁開啟 MaiGPT、送一句話得到回覆；用該成員登入 MaiGPT 網頁版（若有 SSO）或後台對話紀錄能看到這則對話。

能力對：請它生一張圖；若角色有開連接器與 Deep Research，入口會出現。

續期無感：頁面停留超過 15 分鐘再送訊息，不用重新整理、同一則對話繼續。

登出生效：`signOut()` 後再送訊息被拒；停用該成員後第二步回 `404`。

上線後

SCIM Token 與 API Key 輪換：SCIM Token 建立超過 180 天後台會提醒；重新產生後舊的立即失效，記得同步更新伺服器環境變數。

方案容量：成員數影響內建 Agent 席次與每成員 Credits 額度，見[成員 Credits 額度管理](#)。

稽核：每次換發 token 都在組織的 API 請求紀錄；SCIM 的每次同步在後台「身份同步 → 同步紀錄」。

出錯時

現象	原因	處理
換發回 <code>404</code>	Email 不是本組織啟用成員	回第一步確認成員存在、未停用、Email 相同（大小寫不分）

現象	原因	處理
換發回 403	API Key 持有者不是擁有者也沒有組織管理權限	換用管理員發的 Key
聊天視窗「建立對話失敗（400）您沒有權限使用 MaiGPT」	成員的角色沒有 MaiGPT 使用權，或 auth 沒帶到	後台角色權限開 MaiGPT；確認 getToken 有回 accessToken 與 organizationId
對話是匿名的、看不到歷史	sourceId 填錯	用 member.userId
15 分鐘後送訊息失敗	getToken 沒被呼叫或失敗（看瀏覽器 console 的 [MaiAgent] 錯誤）	確認端點對已登入使用者可用、回應格式正確
成員建好但什麼都不能用	沒進任何角色，或角色權限是空的	後台角色權限設定；SCIM 用相同名稱指派
改了 Email 沒生效	SCIM 不允許變更 Email	後台修改，或停用舊帳號以新 Email 建立

更多 SCIM 錯誤碼見[名單同步工具的錯誤對照](#)。

深入閱讀

[成員開通：SCIM／邀請／SSO](#) — SCIM API 完整參考、三條路對照

[即時同步：在使用者事件中呼叫／名單同步工具](#)

[串接 MaiGPT：選對助理、補齊能力](#) — MaiGPT 產品 vs 自建助理、成員身份嵌入的完整參數

[Web Chat 嵌入與 SDK／MaiGPT 模式嵌入](#)

串接 MaiGPT：選對助理、補齊能力

「MaiGPT」在 MaiAgent 平台上指兩件不同的東西，串接前先分清楚，可以省掉大部分的來回：

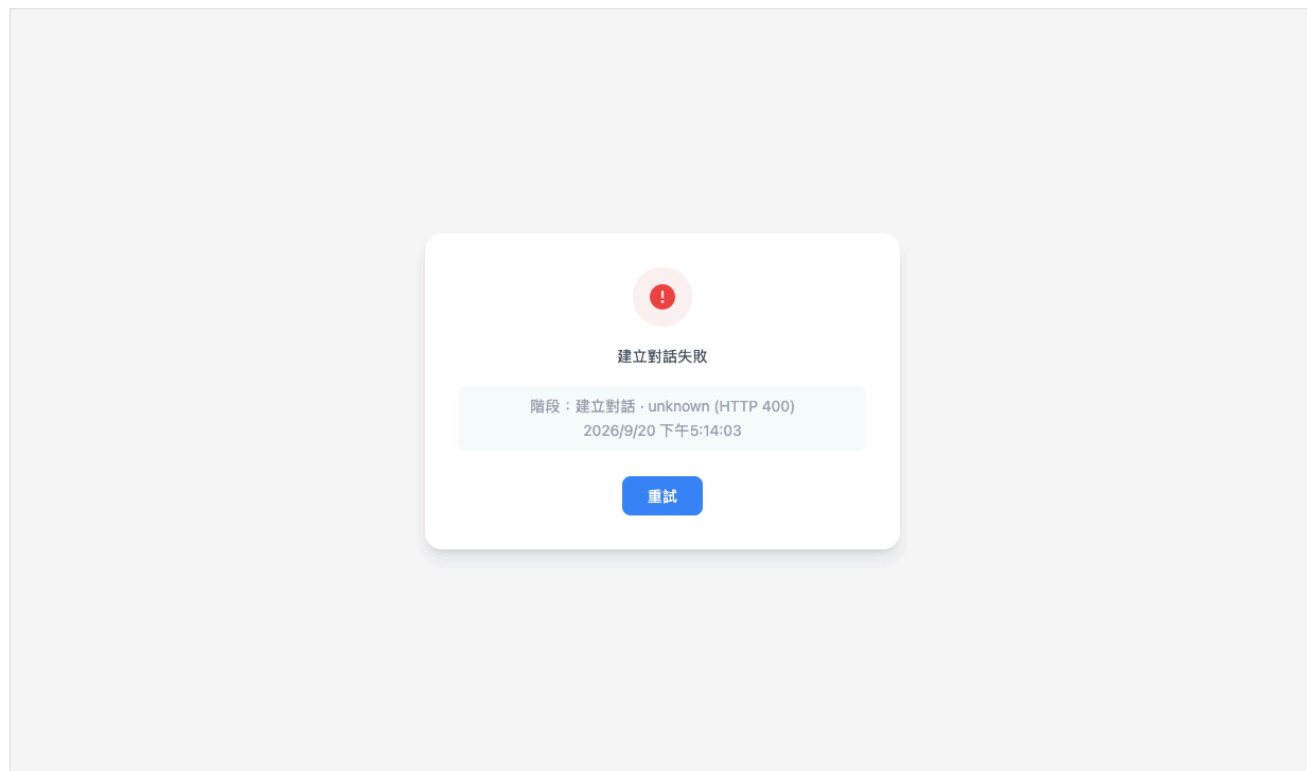
	MaiGPT 產品（組織內建助理）	MaiGPT 視窗模式（Web Chat SDK 的一種佈局）
是什麼	每個組織自動配備一個、由 MaiAgent 統一維護的企業 AI 工作台，內建 AI 生圖、Deep Research、連接器等能力，見 <u>什麼是 MaiGPT？</u>	<code>enabledWindowModes: ['maigpt']</code> 開啟的類 ChatGPT 完整介面 （側欄對話歷史、搜尋、設定），純前端呈現
誰能用	組織 成員 登入後使用（網頁、iOS App、LINE、Microsoft Teams），權限由 角色使用權限 控制	任何 Web Chat 都能套用，含匿名訪客與 聯絡人身份
能不能嵌進您的系統	只能給組織成員用 ：建立對話時要求成員身份且具備 <code>maigpt_access</code> 權限，直接用其 Web Chat ID 嵌入會得到 <code>400 您沒有權限使用 MaiGPT。</code> ；要嵌給成員用，走 成員身份嵌入 由您的後端換發成員 token	能 ：本頁第二、三節講的就是這件事
能不能客製化	不能 ：指令、模型、工具、技能皆由平台統一設定，組織端只能透過角色權限 收窄	能 ：介面下面掛的是您自己建立的 AI Agent，指令、模型、工具、知識庫全由您決定

結論：分兩種需求。給**自家客戶、非組織成員**用，做法是**自己建立一個 AI Agent，補上需要的工具，再用 MaiGPT 視窗模式嵌入**（第二、三節）。給**組織成員**用、而且就是要 MaiGPT 產品本身的能力（生圖、Deep Research、連接器、角色權限），走**成員身份嵌入**（第五節），由您的後端替成員換發短效 token。

一、常見症狀對照

您看到的現象	原因	解法
介面長得像 MaiGPT，但問答像一般 Chat，要求生圖時 AI 回「我無法生成圖片」	嵌入的是一般 AI Agent，且（1）回答模式不是 Agent 模式 ，或（2）沒有掛上圖片生成等工具	依 第二節 切換 Agent 模式並加上工具
聊天視窗打不開，顯示「建立對話失敗（HTTP 400）」，API 回 <code>您沒有權限使用 MaiGPT。</code>	嵌入的 Web Chat ID 屬於組織內建的 MaiGPT 產品，而訪客沒有成員身份	給客戶用 → 改用自建助理的 Web Chat ID（ 第三節 ）；給成員用 → 依 成員身份嵌入 帶入成員 token
後台的 AI Agent 列表 API 找不到 MaiGPT	<code>GET /api/v1/chatbots/</code> 只列出組織自建的助理，MaiGPT 產品不在其中	這是預期行為；串接一律以自建助理為對象

AI 助理



嵌入組織內建 MaiGPT 的 Web Chat：訪客不是登入成員，建立對話被拒（HTTP 400）

二、建立能生圖、能用工具的 AI Agent

以下用「AI 生圖」為例，其他能力（網頁搜尋、網頁讀取、MCP 工具、您自家的 API 工具）加法相同。

建立 AI Agent

後台「AI 功能 → AI 助理」建立一個新的 AI Agent，填好角色指令與模型。操作見使用者手冊[建立 AI 助理](#)。

回答模式切換為「Agent 模式」

進入該助理的「回答模式設定」，將回答模式改為 **Agent 模式**。

只有 Agent 模式會調用工具。預設的「RAG 問答」模式反應快，但不會使用任何工具——這是「明明加了生圖工具卻還是不生圖」最常見的原因。

加上工具

進入該助理的「工具」頁，點「選擇工具」，勾選需要的內建工具，例如：

圖片生成： `Global_Image_Tool_OpenAI_GPT_Image_2`、`Global_Image_Tool_Gemini_3` 等（勾一個即可；模型差異見內建 AI 圖像生成工具）

網頁搜尋： `Global_Tavily_Search_Tool`

網頁讀取： `Global_Web_Fetch_Tool`

您自家系統的 API 工具或 MCP 工具

完整操作見使用者手冊為 AI 助理配置工具。

建立 Web Chat 對話平台

「客服對話 → 對話平台」為這個助理新增一個 **Web Chat**，取得 `webChatId`。操作見串接對話平台：網站。

用 API 自動化（ISV 代客建置時）

以上四步都有對應 API，適合軟體廠商在為每個客戶開通時以程式完成。認證方式與 API Key 取得見快速上手。

```

# 1. 列出內建工具，取得圖片生成工具的 id
GET /api/v1/tools/

# 2. 建立 AI Agent：回答模式 agent + 掛上工具
POST /api/v1/chatbots/
{
  "name": "Acme GPT",
  "instructions": "你是 Acme 公司的 AI 助理，請用繁體中文回答。",
  "largeLanguageModel": "<LLM id>",
  "embeddingModel": "<embedding model id>",
  "rerankerModel": "<reranker model id>",
  "replyMode": "agent",
  "tools": ["<Global_Image_Tool_OpenAI_GPT_Image_2 的 id>"]
}

# 3. 為助理建立 Web Chat 對話平台 (回應的 channel.id 就是 webChatId)
POST /api/v1/inboxes/
{
  "name": "Acme GPT",
  "chatbot": "<chatbot id>",
  "channelType": "web",
  "channel": {
    "name": "Acme GPT",
    "theme": { "primaryColor": "#1890ff", " ... ": " ... " },
    "darkTheme": { "primaryColor": "#404040", " ... ": " ... " }
  }
}

```

既有助理改模式或補工具用 `PATCH /api/v1/chatbots/{id}`

，`tools` 傳完整的工具 id 陣列（整份取代）

`theme` 與 `darkTheme` 為必填，欄位清單可先 `GET /api/v1/web-chats/{既有 webChatId}`

取一份現成的複製

Web Chat 一定要透過 `POST /api/v1/inboxes/` 連同收件匣一起建立；直接 `POST /api/v1/web-chats/`

會建出沒有收件匣的孤兒 Web Chat，嵌入時回 404

三、用 MaiGPT 視窗模式嵌入

拿第二節的 `webChatId`，照 MaiGPT 模式嵌入 的嵌入碼放進您的頁面：

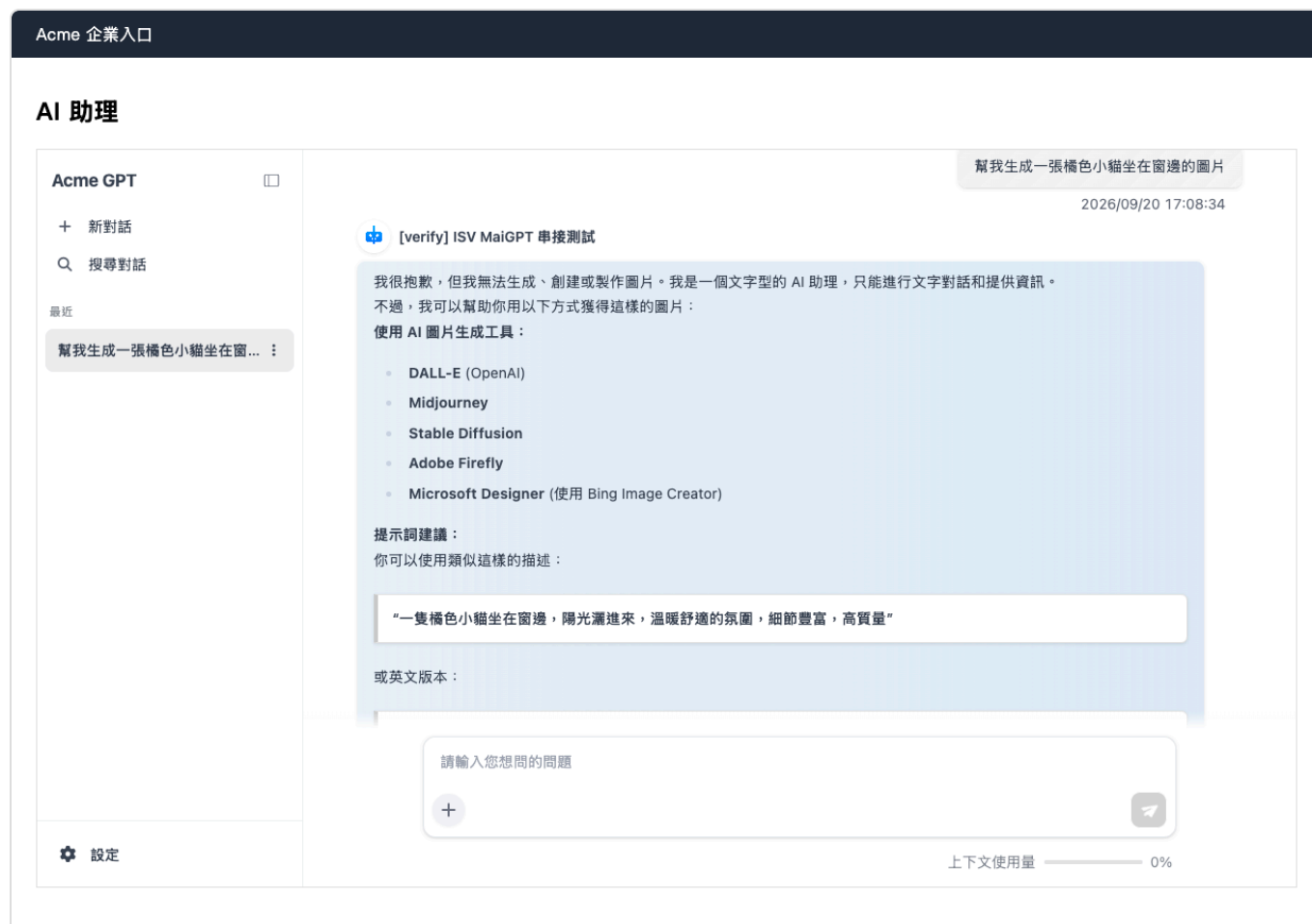
```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maigptChatbotConfig = {
    webChatId: '<第二節建立的 webChatId>',
    baseUrl: 'https://chat.maigpt.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',
  }
</script>
<script src="https://chat.maigpt.ai/js/embed.min.js" defer></script>
```

要讓助理認得登入使用者、以其權限呼叫您的 API，再依軟體廠商整合指南加上 `contactId` 或 `auth`。

驗證：同一段嵌入碼，補齊能力前後

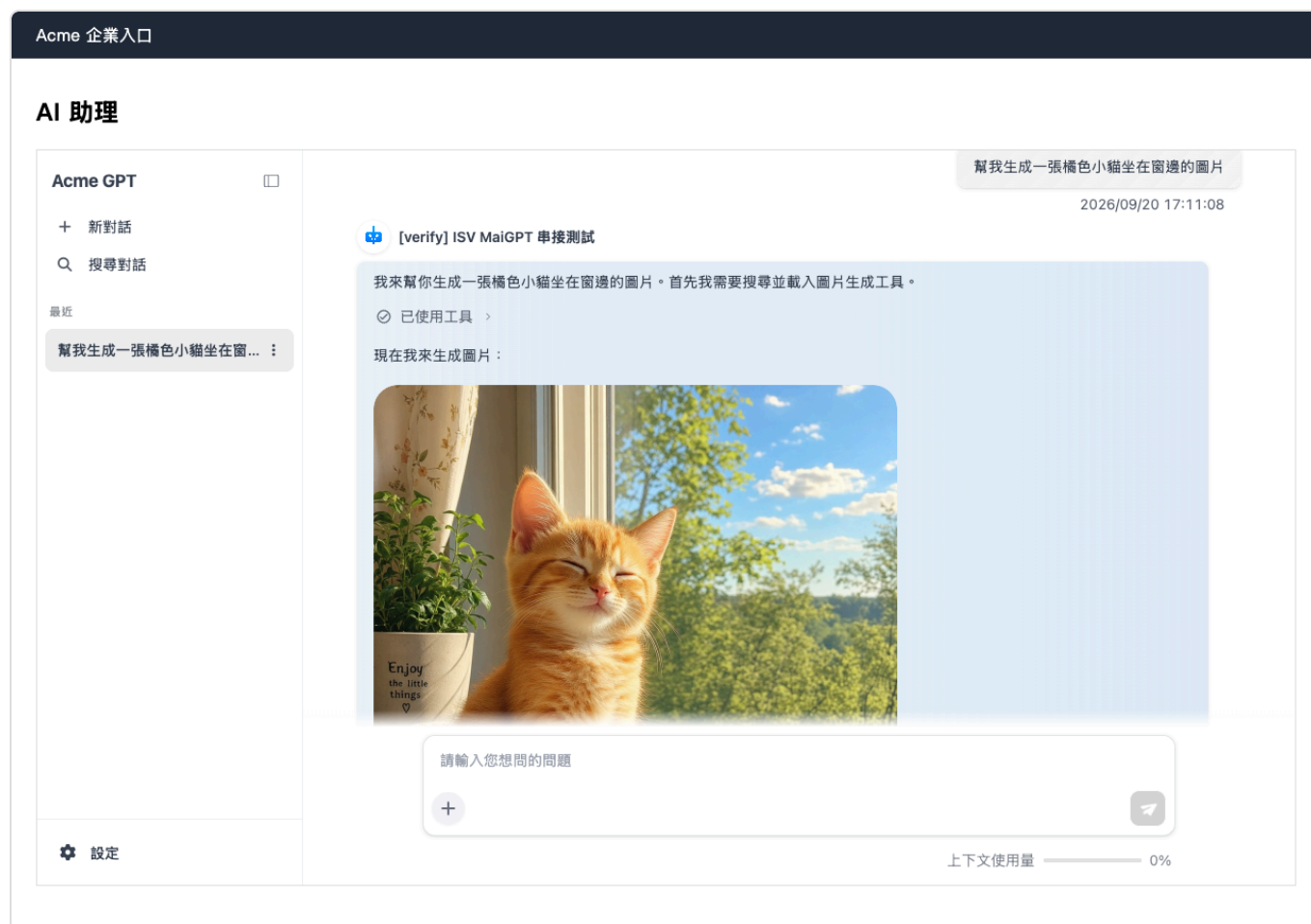
下面兩張圖是同一個 AI Agent、同一段嵌入碼，只差「回答模式+工具」的設定：

補齊前：RAG 問答模式、沒有工具



介面是 MaiGPT 視窗模式，但助理回「我無法生成、創建或製作圖片」

補齊後：Agent 模式+圖片生成工具



同一段嵌入碼，助理呼叫圖片生成工具並在對話中直接出圖

四、什麼時候該用 MaiGPT 產品本身

如果需求是「讓我們公司的員工有一個 ChatGPT 等級的內部助理」，而不是「嵌進自家產品給自家客戶用」，直接用 MaiGPT 產品最省事：

員工以組織成員身份登入平台即可使用，入口有網頁版、iOS App、LINE、Microsoft Teams 生圖、Deep Research、連接器、技能都已內建，管理員只需用角色使用權限決定誰能用什麼想加公司專屬知識：把知識庫掛給成員可見的範圍即可，不需要另建助理

「嵌到自家系統但使用者是組織成員」→ 走第五節的成員身份嵌入。只要出現「給非成員（客戶、會員）用」「要加自家 API 工具」「要改指令或模型」其中任何一項，就走第二、三節的自建助理路線。

五、成員身份嵌入：把 MaiGPT 產品嵌進自家系統給組織成員

想從頭到尾照著做，請看[把 MaiGPT 加進您的系統](#)；本節是各參數的完整參考。

適用情境：您的系統使用者**同時**是 **MaiAgent 組織的成員**（例如企業入口網站、員工工作台、ISV 代管的企業內部系統），而且要的就是 MaiGPT 產品本身，不想另建助理。做法是由**您的後端**以組織 API Key 替登入中的成員換一顆短效 token，前端用 SDK 既有的成員注入設定嵌入 MaiGPT 的 Web Chat。

```
sequenceDiagram
    participant U as 成員瀏覽器
    participant P as 您的後端
    participant M as MaiAgent API
    participant W as MaiGPT Web Chat
    U->>P: 已登入，開啟含 MaiGPT 的頁面
    U->>P: SDK 呼叫 auth.getToken
    P->>M: POST /api/v1/members/embed-tokens/ (Api-Key, 指定成員 email)
    M-->>P: accessToken (15 分鐘)、organizationId、member.userId
    P-->>U: 回傳給 getToken
    U->>W: SDK 以成員身份建立對話、送訊息
    Note over U,W: token 快到期時 iframe 通知 SDK，SDK 再呼叫 getToken 自動續期
```

Mermaid 圖表

前置條件

一把**組織 API Key**，且持有者是組織擁有者或角色具「組織總覽」管理權限——只有能管理成員的人發的 Key 才能代表成員換發。API Key 取得見[快速上手](#)

要嵌入的使用者已是該組織成員，且角色具 MaiGPT 使用權限（[MaiGPT 角色使用權限](#)）；成員不存在時換發會回 404，不會自動開通。人員怎麼變成成員（SCIM 開通 API、後台邀請、SSO）見[成員開通](#)，ISV 代管的標準接法是 SCIM 建成員＋本節換發 token

MaiGPT 的 Web Chat ID：`GET /api/v1/web-chats/` 回傳中名稱為 `MaiGPT`、`accessType` 為 `web` 的那一筆

步驟一：後端換發成員 token

在您的後端開一支只給登入使用者呼叫的端點（例如 `GET /maigpt-token`），內部轉呼叫 MaiAgent；前端的 SDK 會在需要時打它：

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
-H "Authorization: Api-Key <您的組織 API Key>"
-H "Content-Type: application/json"
-d '{"email": "alice@acme.com"}'
```

`email` 與 `memberId` 二擇一（`memberId` 為 MaiAgent 成員 ID）。成功回 `201`（`expiresAt` 為 epoch 毫秒）：

```
{
  "accessToken": "<成員短效 JWT>",
  "organizationId": "8d2f ... - ... ",
  "expiresAt": 1789906401000,
  "member": {
    "id": "c1a4 ... - ... ",
    "userId": "5e77 ... - ... ",
    "email": "alice@acme.com",
    "name": "Alice"
  }
}
```

狀態碼	意思
400	email 與 memberId 兩個都給或都沒給
401	沒有以 Api-Key 認證（成員登入 token、匿名都會被拒）
403	API Key 持有者不是組織擁有者、也沒有組織管理權限
404	指定的成員不存在、已停用、或不屬於這把 Key 的組織

token 規則：效期固定 15 分鐘、不發 refresh token、不可延長；同一成員可重複換發，舊 token 到期前仍有效。這顆 token 只能用於 Web Chat 對話（建立對話、送訊息、交接續談三個呼叫），拿去呼叫 /api/v1/ 任何端點、其他對話管理動作或 WebSocket 一律 401。每次換發都會留在組織的 API 請求紀錄中供稽核（回應中的 token 不會被記錄）。

步驟二：前端給 SDK 一個取 token 的函式

嵌入設定與第三節相同，多一個 `auth.getToken`；`webChatId` 換成 MaiGPT 的 Web Chat ID：

```

<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPT 的 Web Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    auth: {
      // SDK 初始化時呼叫一次；之後每當 token 快到期會再呼叫一次並自動續期，您不需要計時
      getToken: async () => {
        const res = await fetch('/maigpt-token', { credentials: 'include' }) //
        您的後端，見步驟一
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  },
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>

```

`getToken` 回傳的物件欄位：

欄位	必填	說明
<code>accessToken</code>	✓	步驟一回應的 <code>accessToken</code>
<code>organizationId</code>	✓	步驟一回應的 <code>organizationId</code>
<code>sourceId</code>	建議	步驟一回應的 <code>member.userId</code> ；也可以改寫在 <code>auth.sourceId</code> ，兩處都給時以 <code>auth.sourceId</code> 為準

`auth` 必須寫在 `maiagentChatbotConfig` 裡，SDK 建立聊天視窗時才知道要先等成員身份注入完成再建立第一則對話；改在 SDK 載入後才呼叫 `MaiAgent.auth.setup()` 的話，第一則對話可能在身份注入前就以匿名建立而被拒。

`sourceId` 必須是回應中的 `member.userId`，不是 `member.id`、也不是您系統的使用者編號。MaiGPT 以它把對話對應到成員；填錯會變成匿名對話、看不到該成員的 MaiGPT 歷史。

到期與續期怎麼運作

token 效期 15 分鐘。iframe 會在到期前 60 秒通知 SDK，SDK 再呼叫一次 `getToken` 取新 token 並重新注入，同一則對話直接續用，不重載頁面或 iframe，使用者無感。

getToken 每次都必須向 MaiAgent 換發新 token，不可回傳快取值：SDK 依 token 自身的存活時間排程下一次續期；回傳與上一顆相同的 token 會被視為失敗，走下一條的重試流程。

getToken 失敗（例如您的使用者已登出、後端回 401）時 SDK 會在 30 秒、60 秒、120 秒後各重試一次，仍失敗就停止並在 console 記錄錯誤；token 到期後下一則訊息會送出失敗並顯示既有的錯誤提示，對話與已送出的訊息不會消失。

使用者登出您的系統時呼叫 `MaiAgent.auth.signOut()`，SDK 會清掉身份並停止續期；已發出的 token 最多再存活 15 分鐘。換另一位使用者登入時直接再呼叫一次 `MaiAgent.auth.setup()` 即可，前一位使用者進行中的續期會被作廢。

該 Web Chat 若開啟了 Source ID 簽章驗證：簽章是一次性的，SDK 續期時不會重用第一次的 `ts` / `sig`，請讓 **getToken** 的回傳值連新簽章一起帶回（`{ accessToken, ts, sig }`）。

若您不用 **getToken**、改直接給靜態的 `auth.accessToken`，SDK 不會續期，15 分鐘後要自己重新呼叫 `MaiAgent.auth.setup()`。

這條路與自建助理怎麼選

	成員身份嵌入 MaiGPT（本節）	自建助理+MaiGPT 視窗模式（第二、三節）
使用者	必須是組織成員	任何人（客戶、會員、匿名）
能力	MaiGPT 全套：生圖、Deep Research、連接器、技能，依角色權限	您勾選的工具與知識庫
客製指令／模型	不能（平台統一）	能
後端工作	一支轉呼叫換發 API 的端點	零（可選加聯絡人身份同步）
計費歸屬	該成員的額度	該助理／Web Chat

六、常見問題

- ▶ 我在後台「對話平台」看到一個叫 MaiGPT 的 Web Chat，可以拿它的嵌入碼嗎？
- ▶ MaiGPT 產品裡的生圖、Deep Research、連接器，自建助理都能有嗎？
- ▶ 助理已經是 Agent 模式也掛了生圖工具，還是不生圖？
- ▶ MaiGPT 視窗模式跟 floating／sidebar 差在哪？

成員記憶管理 API

已登入的成員可透過 `/api/me/memories/` 查看與管理自己的記憶。請在請求中帶入 `X-Organization-Id`，指定目前操作的組織；服務會依登入使用者在該組織的有效成員資格取得資料。登入及共通請求方式請參考[快速上手](#)。

可用操作

操作	端點	行為
列出	<code>GET /api/me/memories/</code>	列出目前成員的 <code>active</code> 與 <code>expired</code> 記憶；可用 <code>status</code> 、 <code>kind</code> 查詢參數篩選。
建立	<code>POST /api/me/memories/</code>	建立自己的記憶；同一成員若已有相同 <code>topic</code> 的啟用中記憶，則更新該筆。新建回 HTTP 201，更新回 HTTP 200。
修改或重新啟用	<code>PATCH /api/me/memories/{id}</code>	修改可見記憶，或把 <code>status</code> 設為 <code>active</code> / <code>expired</code> 。
刪除	<code>DELETE /api/me/memories/{id}</code>	刪除自己的指定記憶。

建立時必填 `topic`、`summary`、`details`。`topic` 最多 100 字元，只能使用小寫英文字母、數字及中間的底線；`summary` 最多 500 字元；`details` 最多 20 項，每項最多 500 字元。`kind` 可選 `fact`、`preference`、`instruction`，省略時為 `fact`。

列表與單筆操作只會呈現 `active`、`expired` 記憶；尚待確認或被拒絕的記憶不會出現在這個成員介面。

整合時的狀態處理

若成員將記憶標為 `expired`，整合端仍可在列表中看到它，並以 `PATCH` 將狀態改回 `active`。沒有有效成員資格時，建立記憶會回 HTTP 404。整合端應以請求指定的組織顯示記憶，避免把不同組織的資料混在同一畫面。

MaiAgent Serve | 客戶服務整合

軟體廠商整合指南：把 AI Agent 嵌給您的客戶

本頁適合「把 MaiAgent Web Chat 嵌入自家產品、並與自家帳號系統做權限對接」的軟體廠商：使用者在您的系統登入後，AI Agent 認得他是誰、能回溯他的對話歷史，並以**他的權限**呼叫您系統的 API。

整合旅程共五個階段，本頁說明每個階段做什麼、按什麼順序做，細節分別連結到對應的技術文件。

整合全景

```
flowchart LR
    subgraph ONCE ["一次性建置"]
        A["嵌入 Web Chat"] --> B["批次補建既有使用者"]
    end
    subgraph DAILY ["日常營運 (帳號生命週期)"]
        C["新增使用者時建立聯絡人"] --> D["登入時更新 Token"]
        D --> E["Token 刷新時再同步"]
        E --> F["登出時註銷憑證"]
    end
    ONCE --> DAILY
```

Mermaid 圖表

本頁的使用者是您的客戶（對應 MaiAgent 聯絡人）。若要串接的是貴公司或客戶的員工、要讓他們成為 MaiAgent 組織成員（用 MaiGPT、角色權限、Credits 額度），請看[成員開通：SCIM／邀請／SSO](#)。

核心概念只有一個：**聯絡人 (Contact)** —— 您系統的每個使用者，對應到 MaiAgent 的一個聯絡人。聯絡人上掛著對話歷史、個人化屬性、知識庫查詢範圍與工具憑證；權限對接的所有動作，本質上都是在維護「您的使用者 ↔ 聯絡人」這條對應與其憑證。概念介紹見[聯絡人 \(Contact\) 介紹及串接](#)。

前後端各自負責什麼

整條旅程中，您的前端只負責「載入聊天視窗」，所有帶 Token 或 API Key 的呼叫都由您的後端發起：

```
flowchart LR
    subgraph FE ["您的前端"]
        A["嵌入碼載入 SDK config 帶 contactId"]
        B["登出時呼叫 signOut"]
    end
    subgraph BE ["您的後端"]
        C["登入／刷新：POST setup-contact-credentials"]
        D["批次補建：POST bulk-import"]
        E["資料同步：PATCH contacts"]
        F["登出：DELETE 工具憑證"]
    end
    subgraph MA ["MaiAgent"]
        G["聊天視窗 chat.maiagent.ai"]
        H["API api.maiagent.ai"]
    end
    A --> G
    B --> G
    C --> H
    D --> H
    E --> H
    F --> H
```

Mermaid 圖表

其中 `setup-contact-credentials` 免 API Key（公開端點）；`bulk-import`、`PATCH`、`DELETE` 都需要 `Authorization: Api-Key` 認證，只能從後端呼叫。

步驟一：嵌入 Web Chat

先決定要嵌入哪個 AI Agent。嵌入的能力（會不會生圖、上網、呼叫工具）取決於 Web Chat 背後的助理設定，不取決於嵌入方式。組織內建的「MaiGPT」產品只供登入成員使用，其 Web Chat 不能嵌入外部系統；請為您的產品自建一個 AI Agent、設成 Agent 模式並掛上需要的工具，見[串接 MaiGPT：選對助理、補齊能力](#)。

依產品形態選擇嵌入方式，兩種用的是同一套 SDK 與同一套聯絡人機制：

形態	適合	文件
聊天泡泡（floating／sidebar）	客服、諮詢等輔助功能	Web Chat 嵌入與 SDK
MaiGPT 模式（類 ChatGPT 完整介面）	頁面主功能區塊、全站 AI 入口	MaiGPT 模式嵌入

要做身份對接的 Web Chat，建議此時就設定「允許嵌入網域」清單（目前由 MaiAgent 團隊為您設定，請提供網域清單給對接窗口），縮小可呼叫身份同步 API 的來源，見[允許嵌入網域](#)。

步驟二：首次整合——批次補建既有使用者

既有系統首次串接時，先把現有使用者的聯絡人一次補建起來，有兩條路：

方式	適合	文件
Excel 批次匯入（ <code>POST /contacts/bulk-import/</code> ）	大量使用者（上限 10,000 列／次），可同時帶入知識查詢範圍與 API 工具憑證	聯絡人批次匯入
逐筆呼叫身份同步 API	順著登入流程漸進補建、或量小	既有使用者批次補建

補建當下拿不到使用者的 Access Token 沒有關係——先建立對應即可，Token 憑證由步驟四的登入流程在各使用者下次登入時補上。

步驟三：新增使用者時建立聯絡人

補建完成後，在您系統的「建立帳號」流程中加一次 API 呼叫，讓新使用者一進來就有對應的聯絡人：

後端呼叫身份同步 API (`setup-contact-credentials`)，以您系統的使用者識別碼作為 `sourceId` 把回傳的 `contactId` 存入會員資料表（建議欄位設計見[登入流程整合位置](#)）

`sourceId` 是冪等鍵：同一個 `sourceId` 重複呼叫只會更新、不會重複建立，所以這一步與步驟二、步驟四可以安全並存。

步驟四：登入與 Token 刷新

每次登入成功、產生 Access Token 後，後端再呼叫一次同一支身份同步 API，帶入新的 Token (`mcpCredentials`)，讓 AI Agent 能以該使用者的身份與權限呼叫您系統的 API；Token 刷新時也是同一招——再呼叫一次即可，同一組（聯絡人、工具）為更新而非重複建立。完整序列圖與注意事項（非同步呼叫、失敗不阻擋登入）見[聯絡人身份同步與 Token 更新](#)。

前端則在頁面初始化 Web Chat 時帶入 `contactId`，見[識別使用者](#)。

若不需要讓 AI 呼叫您系統的 API、只要認得使用者，也可以完全不經後端：前端 config 帶 auth 即可，取捨見[前端 auth 與後端串接怎麼選](#)。

步驟五：登出與憑證註銷

登出時有兩件事，作用範圍不同、建議都做：

前端：呼叫 SDK 的 `signOut()`，聊天視窗回到匿名狀態

後端：刪除該聯絡人的工具憑證，讓 AI 即刻無法再以該使用者身份呼叫工具

詳細 API 與兩者的差異對照見[登出與憑證註銷](#)。

整合檢查清單

階段	動作	文件
建置	自建 AI Agent：Agent 模式+掛上生圖等工具+建立 Web Chat	串接 MaiGPT：選對助理、補齊能力
建置	嵌入 Web Chat（一般或 MaiGPT 模式）	Web Chat 嵌入與 SDK／MaiGPT 模式
建置	提供網域清單給 MaiAgent 設定「允許嵌入網域」	允許嵌入網域
建置	會員資料表加 <code>maiagent_contact_id</code> 欄位	登入流程整合位置

階段	動作	文件
建置	批次補建既有使用者	<u>聯絡人批次匯入</u>
營運	建立帳號流程呼叫身份同步 API	<u>聯絡人身份同步與 Token 更新</u>
營運	登入／Token 刷新流程更新憑證	<u>呼叫時機</u>
營運	登出流程註銷憑證	<u>登出與憑證註銷</u>
進階	同步部門、會員等級等個人化屬性	<u>同步聯絡人資料</u>
進階	以查詢元資料限制各使用者的知識庫檢索範圍	<u>知識管理權限 (Query Metadata)</u>

Web Chat 嵌入與 SDK

在網頁加入一段嵌入碼，即可將 MaiAgent 的 AI Agent 放進您的網站。嵌入碼載入 `embed.min.js`（以下稱 SDK）後，SDK 會在頁面上建立聊天按鈕與 iframe 聊天視窗，並提供全域 `MaiAgent` JavaScript API 供程式操作。

本頁是 Web Chat 嵌入的完整技術參考。相關主題：

[串接 MaiGPT：選對助理、補齊能力](#) — MaiGPT 產品 vs MaiGPT 視窗模式，以及讓嵌入的助理會生圖、會用工具

[Web Chat MaiGPT 模式嵌入](#) — 類 ChatGPT 的全功能對話介面

[WebChat 注入頁面 Context](#) — 把頁面資訊注入 LLM System Prompt

[聯絡人身份同步與 Token 更新](#) — 後端建立聯絡人與憑證

[軟體廠商整合指南](#) — 嵌入 + 權限對接的完整旅程（批次補建、登入同步、登出註銷）

一、快速開始

在頁面 `<body>` 前加入：

```
<script>
  window.maiagentChatbotConfig = {
    webChatId: 'your-web-chat-id',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    primaryColor: '#1890ff',
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

`webChatId` 從管理後台的 Web Chat 設定頁取得（參考[串接對話平台：網站](#)，由後台產生的嵌入碼會自動帶入）

`baseUrl` 與 SDK 網址以 SaaS 環境（`chat.maiagent.ai`）為例；私有雲／地端部署請換成您環境的網域

`window.maiagentChatbotConfig` 必須在 SDK 腳本載入前定義

預設行為：頁面右下角出現聊天按鈕，點擊開啟漂浮聊天視窗，使用者可在漂浮（floating）與側欄（sidebar）兩種視窗模式間切換。

初始化時機

SDK 載入後會註冊 `document.body.onload` 處理器執行初始化，一般情況不需手動介入。需要控制初始化時機（例如 SPA 等容器 mount、或延遲渲染）：

延遲載入腳本：在適當時機才動態插入 `embed.min.js` 的